

Architecture du projet Congress Trading

Guide complet de la structure — code *et* données, et leur enchaînement

Document dérivé et vérifié sur le **code source réel** (corps des fonctions, arborescence **data/**, en-têtes CSV)
— pas sur les docs antérieures.

En une phrase

Le projet transforme des **déclarations boursières officielles** des membres du Congrès US (PDF lisibles du *House Clerk* + scans + site *eFD* du Sénat) en **une table propre, une ligne par transaction**, par chambre et par an (2020–2026) — **identifiée** (qui est le membre), **enrichie** (ticker, secteur, ancienneté), **dédupliquée**, et **validée** contre une source externe (Quiver). Tout se lance par **une seule commande**.

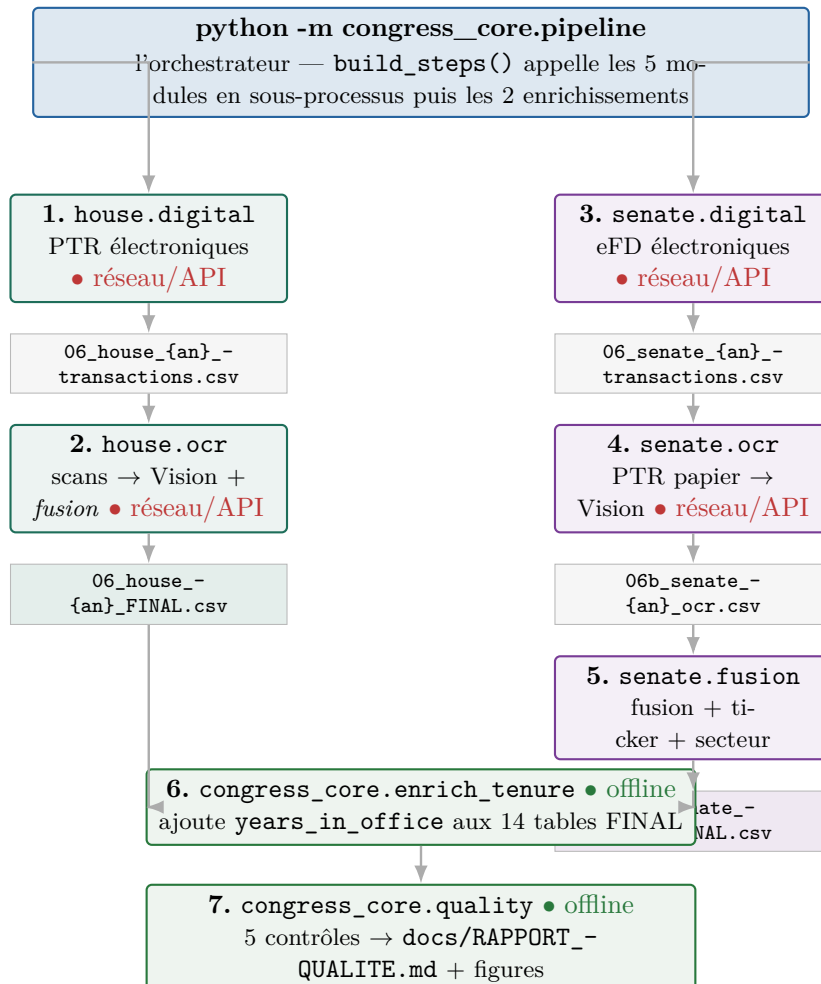
Table des matières

1	Vue d'ensemble — le flux de bout en bout	3
2	La grande idée : 3 couches de code + 1 couche de données	3
3	Le cœur partagé <code>congress_core/</code> — outil par outil	4
3.1	<code>schema.py</code> — la clé d'identité d'une transaction + la dedup	4
3.2	<code>identity.py</code> ★ — nom écrit sur le PDF → <i>quel membre</i> (bioguide)	4
3.3	<code>amounts.py</code> — fourchette de montant → milieu, et code → type d'opération	5
3.4	<code>tickers.py</code> — symbole boursier + type d'actif	5
3.5	<code>vision_ocr.py</code> — lire les scans avec Claude Vision (+ cache)	5
3.6	<code>llm_resolve.py</code> — cache LLM (nom→ticker)	6
3.7	<code>quiver.py</code> — validation externe (jamais réinjectée)	6
3.8	<code>crosscheck.py</code> — statut de validation par déposant	6
3.9	<code>sector_enrich.py</code> — ticker → secteur GICS → ETF SPDR	6
3.10	<code>reporting.py</code> · <code>paths.py</code> — conventions & accès	7
3.11	Les 3 ajouts récents (couche <code>run</code> & livrables)	7
4	Les deux pipelines, étape par étape	7
4.1	House — <code>digital.run_year()</code> puis <code>ocr.run_ocr_year()</code>	7
4.2	Sénat — <code>digital.run_year()</code> → <code>ocr</code> → <code>fusion.main()</code>	8
4.3	Ce qui est COMMUN vs SPÉCIFIQUE (et pourquoi)	9
5	Le voyage d'une transaction	9
6	La couche données — <code>data/</code> (~3 700 fichiers)	9
6.1	Carte des dossiers (comptes réels)	9
6.2	La convention de numérotation <i>est</i> l'enchaînement	10
6.3	Le schéma FINAL (28 colonnes) — colonne, sens, origine	10
7	Concepts transverses (ce qui revient partout)	11

8 Lancer & lire le code	12
Annexe — les 44 fichiers .py	12

1 Vue d'ensemble — le flux de bout en bout

Il existe **un point d'entrée unique**, `congress_core/pipeline.py`, qui enchaîne 7 étapes (commande : `python -m congress_core.pipeline --years 2020-2026`). Les étapes 1–5 produisent les données (et exigent réseau/API) ; les étapes 6–7 sont locales et gratuites.

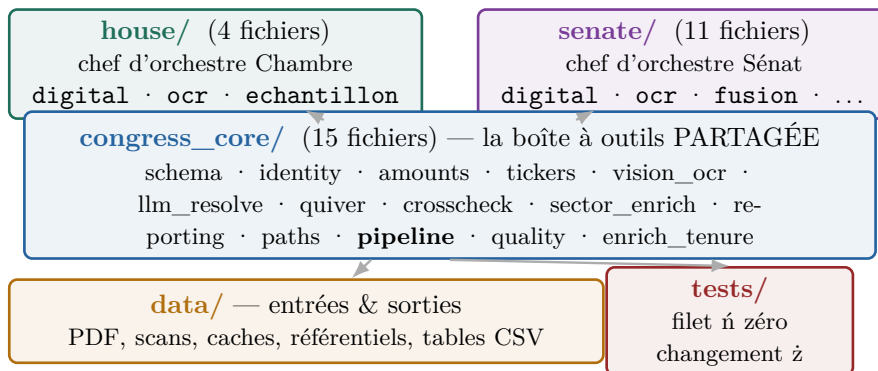


Deux niveaux d'enchaînement qu'il faut distinguer pour ne pas se perdre :

- **Niveau orchestration** (ci-dessus) — les 7 grandes étapes entre modules.
- **Niveau micro-séquence** — à l'intérieur de chaque étape, une suite ordonnée d'appels (ex. `run_year` : `parse` → `identité` → `ticker` → `dédup` → `Quiver`). Détaillé en §4.

2 La grande idée : 3 couches de code + 1 couche de données

La structure n'est pas néplein de fichiers étalés : c'est une **boîte à outils partagée** que **deux chefs d'orchestre** (une par chambre) utilisent, plus la donnée et les tests.



L'analogie qui débloque tout

`congress_core` = les **ingrédients et ustensiles** (couper un nom en bioguide, lire un scan, calculer un secteur...). `house/` et `senate/` = **deux recettes** qui s'en servent dans leur ordre. Elles diffèrent *uniquement* parce que les **sources** diffèrent : House = index XML + PDF du *Clerk* ; Sénat = site *eFD* (HTML) + images .gif. C'est pourquoi `senate/` a plus de fichiers (scraping et OCR papier plus complexes).

3 Le cœur partagé `congress_core/` — outil par outil

Pour chaque module : son rôle (déduit du *code*), puis ses fonctions clés avec **ce qu'elles font** et **ce qu'elles renvoient**.

3.1 `schema.py` — la clé d'identité d'une transaction + la dédup

Définit les ordres de colonnes canoniques et la **clé naturelle** (7 champs, *sans* le ticker → robuste à l'enrichissement). C'est le socle de la déduplication non destructrice.

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>natural_key</code>	Concatène 7 champs (<code>chamber</code> , <code>declarant_name</code> , <code>transaction_date</code> , <code>asset_description</code> , <code>operation_type</code> , <code>amount_range</code> , <code>owner</code>) en une chaîne stable séparée par <code> </code> .	<i>str</i> (clé textuelle)
<code>natural_key_hash</code>	SHA-256 de cette chaîne. Source unique House/Sénat.	<i>str</i> (64 hex)
<code>add_occurrence_index</code>	Numérote les lots identiques au sein d'un même dépôt : <code>cumcount</code> par (<code>doc_id</code> , <code>natural_key_hash</code>).	<i>DataFrame</i> + <i>col occurrence_index</i>
<code>dedup_canonical</code>	Trie par date de divulgation décroissante, garde la 1 ^{re} par (<code>hash</code> , <code>occurrence_index</code>) → retire les doublons cross-dépôt, préserve les lots intra-dépôt.	<i>DataFrame déduplicé</i>

3.2 `identity.py` ★ — nom écrit sur le PDF → *quel membre* (bioguide)

Priorité #1 du projet. Normalise les noms, charge le référentiel *congress-legislators* (live, repli YAML), résout le `bioguide_id`, et enrichit parti / commissions / ancienneté.

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>norm / strip_accents</code>	Minuscule, sans accent, sans ponctuation — pour comparer des noms.	<i>str</i> normalisé
<code>split_name</code>	ñ First Middle Last, Jr z → (<code>last</code> , <code>first</code> , [<code>middles</code>]) ; retire les suffixes.	<i>tuple</i>

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>load_reference</code>	Charge législateurs + commissions, construit les index nom→bioguide, calcule l'année de 1 ^{re} entrée par bioguide.	<i>objet Reference</i>
<code>make_matcher</code>	Renvoie <code>match(last,first)</code> : cascade override → exact → surnom → par-nom-unique.	<i>fonction → bio-guide/None</i>
<code>enrich_identity</code>	Applique le matcher, ajoute <code>bioguide_id</code> , <code>chamber</code> , <code>party</code> , <code>committee_membership</code> , <code>committees_key_flag</code> .	<i>DataFrame + 5 colonnes</i>
<code>add_years_in_office</code>	<code>année(transaction) - année(1^{re} entrée)</code> par bioguide.	<i>DataFrame + years_in_office</i>

Écart code/doc noté par l'audit : le paramètre `chamber_priority` de `make_matcher` est documenté mais **jamais utilisé** dans le corps (réservé). Le code prime.

3.3 `amounts.py` — fourchette de montant → milieu, et code → type d'opération

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>amount_midpoint</code>	Extrait les montants \$ par regex : 2+ → (min+max)/2; 1 → la valeur; 0 → None.	<i>float/None</i>
<code>operation_type_from_code</code>	P→Purchase, E→Exchange, S+partial→Sale (Partial), S+full→Sale (Full), défaut S→Sale.	<i>str</i>

3.4 `tickers.py` — symbole boursier + type d'actif

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>recover_ticker</code>	Si l'intitulé <i>est</i> un ticker (regex 1-5 majuscules, +PUT/CALL).	<i>str/None</i>
<code>explicit_ticker</code>	Cherche un ticker explicite en fin d'intitulé (ń... - AAPLž / ń... (AAPL)ž).	<i>str/None</i>
<code>norm_asset</code>	Normalise l'intitulé pour la recherche par dictionnaire (retire CORP/INC/LLC/ETF...).	<i>str</i>
<code>infer_asset_type</code>	Route selon la chambre : House (Gov Security, Mutual Fund, Corporate Bond, Stock...) vs Sénat (Municipal Security...).	<i>str/None</i>

3.5 `vision_ocr.py` — lire les scans avec Claude Vision (+ cache)

Convertit PDF/images en PNG, **redresse** les pages couchées, extrait via `tool_use` forcé, et **cache** par (`model`, `prompt_sha`) — un re-run ne re-paie rien.

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>pdf_to_b64_images</code>	Rend chaque page en PNG; si <code>deskew</code> , détecte+corrige la rotation.	<i>liste de PNG b64</i>
<code>detect_rotation</code>	Montre la page aux 4 rotations, fait <i>choisir</i> la droite par Vision (robuste).	<i>0/90/180/270</i>
<code>prompt_sha</code>	<code>sha256(prompt + tool + pipeline_tag)[:12]</code> — la version du cache.	<i>str (12 hex)</i>

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>extract_cached</code>	Si cache valide (même <code>prompt_sha+model</code>) → renvoie sans appel API; sinon découpe en lots, ne re-paie que les lots en erreur .	(<i>transactions</i> , <i>objet-cache</i>)

3.6 `llm_resolve.py` — cache LLM (nom→ticker)

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>VersionedLLMCache</code>	<code>.load/.save</code> : lit/écrit un cache JSON { <code>model</code> , <code>prompt_sha</code> , <code>mappings</code> } ; ignore si la version ne correspond pas.	<i>dict de mappings</i>
<code>map_batch</code>	Appel Claude <code>tool_use</code> forcé + backoff sur erreurs réseau.	<i>dict (input du tool)</i>
<code>resolve_names_to_tickers</code>	Cible les lignes <code>ticker_source='none'</code> , complète par LLM les noms absents du cache.	<i>DataFrame (ticker mis à jour)</i>

3.7 `quiver.py` — validation externe (jamais réinjectée)

Quiver Quantitative sert de **vérité-terrain indépendante**. Clé House `ń` brute `ž` vs clé Sénat `ń` normalisée `ž` (différence assumée).

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>fetch_quiver</code>	Lit le cache local s'il existe, sinon l'API (token requis) ; filtre par chambre.	<i>DataFrame/None</i>
<code>validate_quiver_house</code>	Comptes par déposant (nous vs Quiver) + verdicts + recouvrement transaction-niveau (clé <code>bioguide ticker date type</code>).	(<i>comparaisons</i> , <i>vrais-absents</i> , <i>stats</i>)
<code>reconcile</code>	Version Sénat normalisée → tables 07c–07f (recouvrement, accord de champs, deltas).	<i>dict de DataFrames</i>

3.8 `crosscheck.py` — statut de validation par déposant

Matérialise le constat : Quiver/Kadoa/HSW sont **aveugles au papier** → notre OCR est la source unique.

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>per_filer_status</code>	Par bioguide : total, part OCR, nb docs, comptes Quiver → statut <code>quiver_validable</code> / <code>ocr_unique</code> / <code>digital</code> .	<i>DataFrame par déposant</i>
<code>add_external_counts</code>	Ajoute les comptes Kadoa / House Stock Watcher par nom (prouve l'isolement des OCR-uniques).	<i>DataFrame + colonnes</i>
<code>summary</code>	Agrège par statut (déposants, transactions, dont OCR).	<i>DataFrame récap</i>

3.9 `sector_enrich.py` — ticker → secteur GICS → ETF SPDR

Résolution hybride : **yfinance** (factuel) → repli **LLM** → **overrides** manuels, avec cache d'audit (qui a tranché : `yf` vs `llm`).

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>resolve_yfinance</code>	Secteur via <code>yfinance.Ticker(t).info['sector']</code> , mappé en GICS, plusieurs passes.	<i>{ticker : secteur/None}</i>
<code>resolve_llm</code>	Repli LLM <code>tool_use</code> (enum GICS strict) si <code>yfinance</code> échoue; dégrade si pas de clé API.	<i>{ticker : secteur/None}</i>
<code>resolve_sectors</code>	Orchestration : <code>cache</code> → <code>yfinance</code> → LLM → audit.	<i>{ticker : {yf, llm, sector, source}}</i>
<code>enrich_sectors</code>	Ajoute <code>sector_gics</code> , <code>etf_proxy</code> (table GICS→ETF), <code>sector_source</code> + overrides.	<i>DataFrame + 3 colonnes</i>

3.10 reporting.py · paths.py — conventions & accès

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>reporting.qa_flags</code>	Signale les lignes où un champ obligatoire est vide.	<i>DataFrame des manques</i>
<code>reporting.upsert_status</code>	Met à jour un tableau de bord CSV (remplace les lignes d'une clé).	<i>DataFrame fusionné</i>
<code>paths.DataRoot</code>	Encapsule <code>data/{chambre}/</code> : <code>.tables</code> , <code>.pdfs</code> , <code>.reference</code> , <code>.ocr_cache</code> , <code>.reports...</code>	<i>objets Path</i>
<code>paths.get_secret / load_env</code>	Récupère <code>ANTHROPIC_API_KEY</code> (env puis <code>.env</code>).	<i>str/None</i>

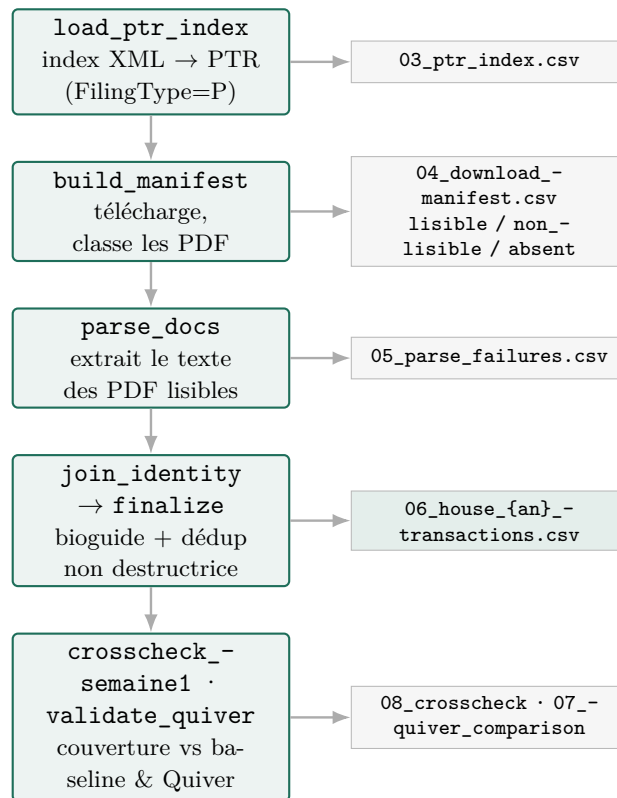
3.11 Les 3 ajouts récents (couche n run z & livrables)

Fonction	Ce qu'elle fait (corps réel)	Ce qu'elle renvoie
<code>pipeline.build_steps</code>	Construit la séquence des 7 étapes (selon <code>--skip-ocr</code> , <code>--force...</code>).	<i>liste (label, args)</i>
<code>pipeline.main</code>	Lance chaque étape en sous-processus ; <code>--dry-run</code> imprime sans exécuter.	—
<code>quality.*</code> (14 fn)	Charge les FINAL, calcule les 5 contrôles (cohérence dates, délai 45 j, distribution montants, coverage/membre, achats sans sortie) → <code>RAPPORT_QUALITE.md</code> + figures.	<i>rapport + PNG</i>
<code>enrich_tenure.enrich_file</code>	Append <i>byte-préservant</i> de <code>years_in_office</code> (idempotent).	<i>bool (modifié ?)</i>

4 Les deux pipelines, étape par étape

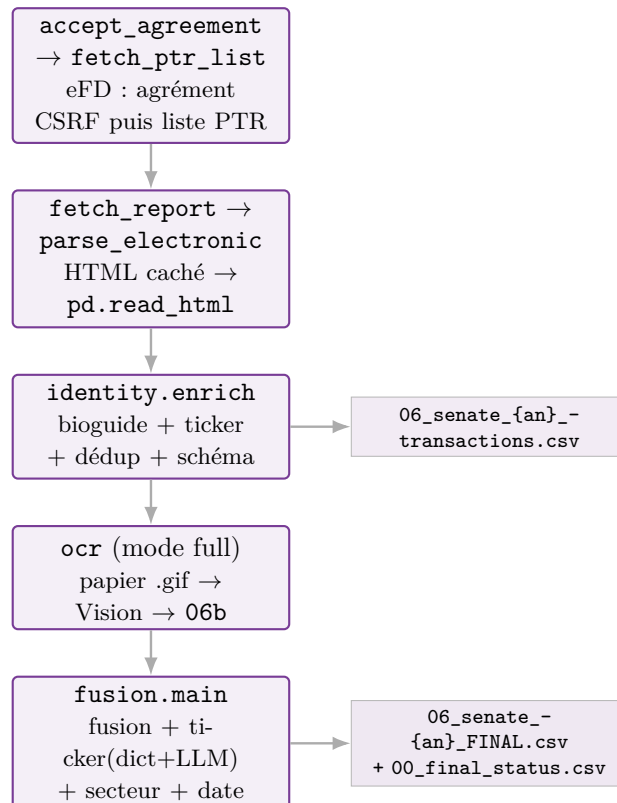
4.1 House — `digital.run_year()` puis `ocr.run_ocr_year()`

Séquence **réelle** (lue dans le corps de `run_year`, `house/digital.py`) :



Puis `house.ocr.run_ocr_year()` reprend le `04_download_manifest` (bucket `non_lisible`) → **Vision** (`extract_cached`, cache versionné) → ticker (dict + LLM) → identité → `occurrence_index` → dédup → **fusion digital+OCR** → secteur GICS → écrit `06_house_{an}_FINAL.csv`. *Côté House, la fusion finale vit dans `ocr.py`.*

4.2 Sénat — `digital.run_year()` → `ocr` → `fusion.main()`



4.3 Ce qui est COMMUN vs SPÉCIFIQUE (et pourquoi)

	House	Sénat
Source	index XML + PDF (<i>Clerk</i>)	site eFD : HTML + images .gif
Acquisition	téléchargement PDF	scraping + agrément CSRF (accept_agreement)
Parsing digital	pdfplumber + regex	pd.read_html (tableaux HTML)
Fusion finale	dans ocr.py	module dédié fusion.py

Identique pour les deux chambres (fourni par congress_core)

identité bioguide · clé natural_key_hash + occurrence_index · montants/midpoint · OCR Vision (cache prompt_sha) · ticker (dict+LLM) · secteur GICS→ETF · validation Quiver · schéma FINAL 28 colonnes.

5 Le voyage d'une transaction

Comment une ligne brute devient une ligne de la table FINAL — en nommant la fonction à chaque étape.



6 La couche données — data/ (~3 700 fichiers)

Rangée par chambre, puis par an. Tout ce qui n'est pas une sortie est une entrée ou un cache.

6.1 Carte des dossiers (comptes réels)

Dossier	Contenu	Rôle (qui écrit / qui lit)
house/pdfs/{an}	547 PDF	sources scannées brutes ; lues par <code>house.ocr</code>
house/index/	7 XML	index annuel du <i>Clerk</i> ; lu par <code>load_ptr_index</code>
house/ocr_cache/{an}	546 JSON	cache Vision (1 par doc, clé <code>prompt_sha</code>) ; écrit/lu par <code>vision_ocr</code>
house/reference/	4 YAML + 1 CSV	annuaire élus + commissions + baseline ; lu par <code>identity</code>
house/tables/{an}	14 CSV/an	toutes les sorties (voir convention ci-dessous)
house/tables/	8 CSV	caches & tableaux de bord (<code>_quiver_house_cache</code> , <code>00_*</code> , <code>census</code>)
senate/{an}	9 CSV/an	sorties Sénat (06, 06b, 06_FINAL, 07*, 05)
senate/ocr_cache/	130 JSON	cache Vision papier Sénat
senate/reports/	935 HTML	pages eFD cachées ; <code>reports/media/</code> = 352 .gif (scans)
senate/reference/	4 YAML + 1 CSV	référentiel Sénat (miroir House)
external/senate_openset/	62 JSON	données externes (ex. <code>kadoa_filers.json</code>) pour le cross-check

6.2 La convention de numérotation *est* l'enchaînement

Sur le disque, le préfixe d'un fichier = son étape dans le pipeline.

Préfixe	Étape	Écrit par
03_	index des PTR (FilingType=P) dans la fenêtre	<code>house.digital</code>
04_	manifeste : lisible / non_lisible / absent	<code>house.digital</code>
05_	échecs de parsing	<code>house & senate digital</code>
06_	table digitale (PTR électroniques)	<code>*.digital</code>
06b_	table OCR (scans/papier)	<code>*.ocr</code>
06c_	échecs OCR	<code>*.ocr</code>
06_..._FINAL	digital + OCR fusionnés , 28 colonnes	<code>house.ocr / senate.fusion</code>
07_...07f_	comparaison/réconciliation Quiver (validation)	<code>quiver / quiver-audit</code>
08_	cross-check n° semaine 1 z (House)	<code>house.digital</code>
00_	tableaux de bord (statut, concordance)	<code>.digital / fusion</code>
_préfixe	caches & brouillons (<code>_quiver_*_cache</code> , <code>_scan_census...</code>)	divers

6.3 Le schéma FINAL (28 colonnes) — colonne, sens, origine

#	Colonne	Sens	Remplie par
1	<code>bioguide_id</code>	identifiant officiel du membre	<code>identity</code>
2	<code>declarant_name</code>	nom tel que déposé	<code>parse / OCR</code>
3	<code>chamber</code>	house / senate	<code>enrich_identity</code>
4	<code>party</code>	parti	<code>Reference.party</code>
5	<code>state_district</code>	état + district	<code>03_ptr_index</code>
6	<code>committee_membership</code>	commissions (liste)	<code>Reference.committees</code>
7	<code>committees_key_flag</code>	commission n° clé z ?	<code>is_key_committee</code>

#	Colonne	Sens	Remplie par
8	transaction_date	date de la transaction	parse / OCR
9	disclosure_date	date de divulgation (dépôt)	03_ptr_index
10	ticker	symbole boursier	tickers / llm_resolve
11	asset_description	libellé de l'actif	parse / OCR
12	asset_type	type d'actif	infer_asset_type
13	sector_gics	secteur GICS	sector_enrich
14	etf_proxy	ETF SPDR du secteur	GICS_TO ETF
15	operation_type	Purchase / Sale / ...	operation_type_from_code
16	amount_range	fourchette déclarée	parse / OCR
17	amount_midpoint	milieu de la fourchette	amount_midpoint
18	amount_split_flag	lot multi-actifs?	finalize
19	owner	propriétaire (self/spouse...)	normalize
20	doc_id	identifiant du PTR	03_ptr_index
21	source_url	URL de la source	03_ptr_index
22	natural_key_hash	clé de dédup (7 champs)	schema
23	occurrence_index	rang du lot intra-dépôt	add_occurrence_index
24	provenance	*-pdf/efd-electronic / -ocr	pipeline
25	date_confidence	plausible / implausible (75 j)	fusion.date_confidence
26	ticker_source	explicit / elec_dict / llm / none	tickers / llm
27	sector_source	yfinance / llm / manual	sector_enrich
28	years_in_office	ancienneté à la date du trade	enrich_tenure

Les 12 premiers champs *í métier* sont garantis présents; *ticker/sector_gics* restent parfois vides *par nature* (actifs non cotés : obligations, munis).

7 Concepts transverses (ce qui revient partout)

natural_key_hash + occurrence_index — la dédup non destructrice. La clé (7 champs, sans ticker) identifie une transaction; l'occurrence_index numérote les **lots réels identiques** d'un même dépôt (ex. Khanna déclarant 3 fois le même titre via 3 comptes). On déduplique sur le *couple*, ce qui retire les doublons cross-dépôt sans écraser les lots légitimes.

provenance — d'où vient la ligne. house-pdf-electronic / house-pdf-ocr / senate-efd-electronic / senate-efd-ocr. C'est ce qui distingue digital et OCR dans la table FINAL.

date_confidence — garde-fou de date. plausible si le délai dépôt–transaction $\in [0, 75]$ jours, si non implausible (surtout des dates manuscrites OCR ambiguës).

ticker_source / sector_source — traçabilité. Chaque enrichissement dit *comment* il a été obtenu (explicit/elec_dict/llm/none; yfinance/llm/manual) → auditable.

Cache versionné (prompt_sha). Tous les appels coûteux (Vision, LLM) sont cachés par (model, prompt_sha). Tant que le prompt ne change pas, un re-run ne re-paie rien — c'est ce qui rend le pipeline **incrémental**.

8 Lancer & lire le code

La commande unique

```
python -m congress_core.pipeline --years 2020-2026
--dry-run (voir la séquence sans rien exécuter) · --skip-ocr (digital seul) · --force (ignorer le
cache Vision).
```

Vérifier que rien n’a bougé (filet à zéro changement) : `python tests/regression/check_golden.py` (House) et `senate_check_golden.py` (Sénat) doivent afficher à ZÉRO ÉCART.

Index à étape → où lire :

Étape	Fichier : fonction
Orchestration	<code>congress_core/pipeline.py</code> : <code>build_steps</code> / <code>main</code>
House digital	<code>house/digital.py</code> : <code>run_year</code>
House OCR + FINAL	<code>house/ocr.py</code> : <code>run_ocr_year</code>
Sénat digital	<code>senate/digital.py</code> : <code>run_year</code>
Sénat OCR	<code>senate/ocr.py</code> : <code>build_table</code> <code>senate/ocr_engine.py</code> : <code>extract_report</code>
Sénat FINAL	<code>senate/fusion.py</code> : <code>main</code>
Identité	<code>congress_core/identity.py</code> : <code>make_matcher</code> / <code>enrich_identity</code>
Ticker	<code>congress_core/tickers.py</code> + <code>senate/ticker_resolve.py</code>
Secteur	<code>congress_core/sector_enrich.py</code> : <code>enrich_sectors</code>
Validation	<code>congress_core/quiver.py</code> + <code>senate/quiver_audit.py</code>
Ancienneté	<code>congress_core/enrich_tenure.py</code>
Rapport qualité	<code>congress_core/quality.py</code>

Annexe — les 44 fichiers .py en un coup d’œil

Fichier	Rôle
congress_core/ (15)	
<code>schema.py</code>	clé naturelle + dédup
<code>identity.py</code>	nom → bioguide + parti/commissions/ancienneté
<code>amounts.py</code>	fourchette → milieu, code → opération
<code>tickers.py</code>	symbole + type d’actif
<code>vision_ocr.py</code>	OCR Claude Vision + cache + des skew
<code>llm_resolve.py</code>	cache LLM versionné (nom→ticker)
<code>quiver.py</code>	validation externe (House brute / Sénat normalisée)
<code>crosscheck.py</code>	statut par déposant (OCR-unique vs Quiver)
<code>sector_enrich.py</code>	ticker → GICS → ETF (yfinance+LLM)
<code>reporting.py</code>	conventions de sortie, QA, Excel
<code>paths.py</code>	chemins (DataRoot) + secrets
<code>pipeline.py</code>	l’orchestrateur (7 étapes)
<code>quality.py</code>	rapport de qualité (5 contrôles)
<code>enrich_tenure.py</code>	years_in_office (append byte-préservant)
<code>__init__.py</code>	version/exports du package
house/ (4)	
<code>digital.py</code>	PTR électroniques → 06_..._transactions
<code>ocr.py</code>	scans → Vision → fusion → 06_..._FINAL
<code>echantillon.py</code>	mesure stratifiée OCR (audit)

Fichier	Rôle
<code>__init__.py</code>	exports
senate/ (11)	
<code>digital.py</code>	eFD électronique → 06..._transactions
<code>ocr.py</code>	papier eFD → Vision → 06b
<code>ocr_engine.py</code>	moteur OCR figé (Q1) réutilisé
<code>fusion.py</code>	fusion digital+OCR + enrichissement → FINAL
<code>identity.py</code>	référentiel + matcher Sénat
<code>ticker_resolve.py</code>	ticker en 3 étapes (explicit/dict/LLM)
<code>sector_enrich.py</code>	secteur GICS→ETF (Sénat)
<code>quiver_audit.py</code>	réconciliation Quiver (07c-07f)
<code>revalidate_-quiver.py</code>	re-validation Quiver offline
<code>census_probe.py</code>	Phase 0 : census + sondage parser
<code>__init__.py</code>	exports
tests/regression/ (14)	
golden (build_golden/check_golden + Sénat) · reproductions (test_schema, test_identity, test_amounts_tickers, test_senate_repro, test_vision_sha, test_tenure, test_incremental, test_crosscheck) · audit_metrics · _apply_house_sector	